KRISHISYNC AI | VIBE CODING SERIES | 4 OF 5

Decision Algorithms

Anti-glut, irrigation, energy and advisory logic with tests and prompts

Market-Aware Agri-Coordination & Smart Energy Optimization for Indian Smallholders

Challenge 1: Sustainable Agriculture & Resource Optimization

Decision Algorithms

These are the brains of KrishiSync. Each is a **pure function**: inputs in, decision out, no database calls inside. All numbers below are **proposed defaults** for a prototype and must be moved to config and tuned with real agronomic and market data.

1. Anti-Saturation (anti-glut) score

For a farm and a candidate crop, look at all planned plantings of that crop within 20 km that will reach harvest in an overlapping window, and compare against what the region can absorb.

```
planned_area(c) = sum(area_ha of crop_plans for crop c within radius R,
                      harvest window overlapping this farm's window)
capacity_area(c) = regional_absorb_t(c) / yield_t_per_ha(c)
saturation(c) = planned_area(c) / capacity_area(c)

band: saturation < 0.60 -> GREEN
      0.60 <= s < 0.85 -> AMBER
      s >= 0.85 -> RED
```

SQL

```
-- PostGIS radius aggregate (R = 20000 m)
SELECT cp.crop_id, SUM(cp.area_ha) AS planned_ha
FROM crop_plans cp JOIN farms f ON f.id = cp.farm_id
WHERE ST_DWithin(f.location, ST_MakePoint(:lon, :lat)::geography, 20000)
  AND cp.expected_harvest_date BETWEEN :win_start AND :win_end
GROUP BY cp.crop_id;
```

2. Crop recommendation ranking

```
score(c) = 0.35 * (1 - min(saturation(c), 1))
         + 0.25 * suitability(c, soil, terrain, season)
         + 0.25 * margin_norm(c) # price trend x yield, scaled 0-1
         + 0.15 * water_efficiency(c) # 1 - water_need / max_water_need

Exclude: RED crops, and crops outside the sowing season.
Return top 3 with a reason code per winning factor (for i18n text).
```

Suitability is a lookup: each crop lists acceptable soil types, slope range and drainage class. Score 1.0 for a full match, 0.5 for a partial match, 0 to exclude. Waterlogging-prone terrain (valley, low slope, poor drainage) excludes high-standing-water crops.

3. Irrigation scheduler with weather pause

```
def plan_irrigation(m, crop_stage, forecast, tariff, pump, now):
    lo, hi, crit = crop_stage.lower, crop_stage.upper, crop_stage.critical
    if m >= hi:
        return NoAction("moisture_ok")
    if forecast.rain_prob_6h >= 0.60 and forecast.rain_mm_6h >= 5:
        return Pause("rain_expected") # unless m < crit
    if m < crit:
        return RunNow("critical_dry") # safety override
    if m < lo:
        slot = next_solar_window(now) if pump.solar else next_off_peak(now, tariff)
        return Schedule(slot, duration_min(m, hi, crop_stage, pump))
    return NoAction("in_band")

volume_L = (hi - m)/100 * root_depth_mm * area_m2 # 1 mm over 1 m2 = 1 L
duration_min = volume_L / (pump.flow_lpm * pump.efficiency)
```

4. Energy: off-peak share and vampire load

- **Off-peak share** = pump kWh in off-peak and solar slots divided by total pump kWh over the period. Show as a percentage.
- **Vampire (idle) load flag:** if pump_on is false and watts stays above a threshold (default 5 W) for 30 minutes or more, raise an amber alert 'idle_draw'.
- **Load balancing:** when several farms on one feeder share the same start time, spread start times across the off-peak window in 10-minute steps. Prototype this as a simple round-robin.

5. N-P-K advisory

Compare latest N, P, K readings to a per-crop, per-stage target range stored in config. Output one of low, ok, high for each nutrient and a reason code, for example 'n_low_top_dress'. Advice is only given where the reading is outside range; applying only what is needed is how the deck's fertiliser-saving target is pursued.

Sensor note: low-cost capacitive probes measure moisture well; N-P-K from cheap probes is rough. Treat N-P-K as indicative and say so in the UI.

Unit tests to write first

Function	Case	Expected
saturation	planned 0 ha	0 and GREEN
saturation	planned = capacity	1.0 and RED
band	exactly 0.60 and 0.85	AMBER and RED
plan_irrigation	moisture above upper	NoAction
plan_irrigation	rain 70%, 8 mm, moisture low but above critical	Pause
plan_irrigation	rain forecast but moisture critical	RunNow
plan_irrigation	solar pump, low moisture at night	Schedule at next solar window
vampire	pump off, 9 W for 45 min	idle_draw alert
recommend	all candidates RED	Empty list plus 'no_safe_crop' reason

Algorithm Vibe Prompts

Paste the Master Prompt first. Always ask for tests *before* implementation so the AI cannot quietly bend the rules to pass.

VIBE PROMPT 1 | Config and data classes

```
Create backend/app/config.py with typed dataclasses holding every threshold and weight:
radius_m=20000, amber=0.60, red=0.85, ranking weights (0.35/0.25/0.25/0.15), rain_prob=0.60,
rain_mm=5, idle_watts=5, idle_minutes=30, tariff slots, and per-crop-stage moisture bands
(lower/upper/critical). Add a loader that reads overrides from env or YAML. No logic yet.
```

VIBE PROMPT 2 | Anti-glut engine, tests first

```
In backend/app/engines/antiglut.py implement saturation(), band() and recommend() as pure
functions exactly as specified in the Algorithms PDF. First write pytest tests from the
test table (including boundary values 0.60 and 0.85), show them failing, then implement
until they pass. Reason codes must be i18n keys like 'low_local_supply'. Do not query the
database inside these functions; take plain data in and return plain data out.
```

VIBE PROMPT 3 | Irrigation scheduler

```
In engines/irrigation.py implement plan_irrigation() with the rule order: no action if above
upper band, rain pause, critical override, then schedule into the next solar window (solar
pumps) or the next off-peak slot (grid pumps). Compute duration from volume and pump flow.
Return a typed result (NoAction | Pause | RunNow | Schedule) with a reason key. Tests first.
```

VIBE PROMPT 4 | Energy and vampire load

```
In engines/energy.py implement off_peak_share(readings, tariff) and detect_idle_draw(readings)
using a rolling window. Add a round-robin start-time spreader for farms sharing a feeder.
Provide a small simulation script that generates 24h of readings with one injected idle-draw
fault and prints what the engine flags.
```

VIBE PROMPT 5 | Advisory and wiring to the API

```
Implement engines/advisory.py for N-P-K low/ok/high with reason codes from a per-crop,
per-stage target table in config. Then wire all engines into the FastAPI routes from the
Architecture PDF: routes load data, call the pure functions, and persist alerts. Add an
end-to-end test using the seed data that proves the over-planted crop turns RED.
```

VIBE PROMPT 6 | Explainability

```
For every recommendation and alert, return a machine-readable 'why' object: the inputs
used, the numbers, and the rule that fired (for example saturation=0.91 vs threshold 0.85).
Add a debug endpoint that shows it. We will use this in the demo to show the system is
transparent, not a black box.
```